Start coding or generate with AI.

Task 1: Data Handling (LO2)

The India air quality dataset (CSV) was downloaded from Moodle and imported into Google Colab. An initial inspection is performed to understand the dataset structure and quality. If multiple CSV files are provided, they are merged into a single dataset to enable comprehensive analysis.

```
from google.colab import files
uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Assessment Data-20251202.zip to Assessment Data-20251202 (1).zip
```

```
import zipfile
import os

zip_path = "/content/Assessment Data-20251202 (1).zip"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("/content/data")

print("Files extracted to /content/data")
```

```
Files extracted to /content/data
```

```
import zipfile
import os

zip_path = "/content/Assessment Data-20251202 (1).zip"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("/content/data")

print("Files extracted to /content/data")
```

```
Files extracted to /content/data
```

```
import glob

csv_files = glob.glob("/content/data/**/*.csv", recursive=True)
print("Number of CSV files found:", len(csv_files))

# show first 10 files as evidence
csv_files[:10]
```

```
Number of CSV files found: 26
['/content/data/Aizawl_data.csv',
 '/content/data/Gurugram_data.csv',
 '/content/data/Jorapokhar_data.csv',
 '/content/data/Mumbai_data.csv',
 '/content/data/Guwahati_data.csv',
 '/content/data/Bengaluru_data.csv',
 '/content/data/Lucknow_data.csv',
 '/content/data/Jaipur_data.csv',
 '/content/data/Chennai_data.csv',
 '/content/data/Kochi_data.csv']
```

```
import pandas as pd
import os

dfs = []

for f in csv_files:
    df = pd.read_csv(f)
```

```
    # standardize column names
    df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]

    # ensure city column exists
    if "city" not in df.columns:
        df["city"] = os.path.basename(f).replace("_data.csv", "")

    df["source_file"] = os.path.basename(f)
    dfs.append(df)

air_quality_df = pd.concat(dfs, ignore_index=True, sort=False)

print("Merged dataset shape:", air_quality_df.shape)
air_quality_df.head()
```

```
Merged dataset shape: (29531, 17)
```

	city	date	pm2.5	pm10	no	no2	nox	nh3	co	so2	o3	benzene	toluene	xylene	aqi	aqi_bucket	source_fi
0	Aizawl	11/03/2020	32.69	47.91	6.99	2.85	11.93	26.64	0.60	4.53	4.48	0.03	0.30	NaN	NaN	NaN	Aizawl_data.c
1	Aizawl	12/03/2020	31.21	38.66	7.20	1.27	10.65	25.63	0.56	4.22	2.81	0.01	0.08	NaN	52.0	Satisfactory	Aizawl_data.c
2	Aizawl	13/03/2020	38.39	46.68	7.19	0.91	10.37	29.16	0.57	4.46	0.18	0.00	0.00	NaN	60.0	Satisfactory	Aizawl_data.c
3	Aizawl	14/03/2020	43.23	50.83	7.14	1.07	10.48	28.95	0.57	4.53	0.41	0.00	0.00	NaN	62.0	Satisfactory	Aizawl_data.c
4	Aizawl	15/03/2020	33.82	41.03	7.09	0.36	9.73	28.41	0.48	4.63	0.30	0.00	0.00	NaN	70.0	Satisfactory	Aizawl_data.c

```
air_quality_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 17 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   city         29531 non-null  object
 1   date         29531 non-null  object
 2   pm2.5        24933 non-null  float64
 3   pm10         18391 non-null  float64
 4   no           25949 non-null  float64
 5   no2          25946 non-null  float64
 6   nox          25346 non-null  float64
 7   nh3          19203 non-null  float64
 8   co           27472 non-null  float64
 9   so2          25677 non-null  float64
 10  o3           25509 non-null  float64
 11  benzene      23908 non-null  float64
 12  toluene      21490 non-null  float64
 13  xylene       11422 non-null  float64
 14  aqi          24850 non-null  float64
 15  aqi_bucket   24850 non-null  object
 16  source_file  29531 non-null  object
dtypes: float64(13), object(4)
memory usage: 3.8+ MB
```

```
air_quality_df.isna().sum().sort_values(ascending=False).head(15)
```

	0
xylene	18109
pm10	11140
nh3	10328
toluene	8041
benzene	5623

```
air_quality_df.duplicated().sum()
```

```
np.int64(0)
```

```
air_quality_df.to_csv("/content/india_air_quality_merged.csv", index=False)
print("Saved merged dataset to /content/india_air_quality_merged.csv")
```

```
Saved merged dataset to /content/india_air_quality_merged.csv
```

```
air_quality_df.to_csv("/content/india_air_quality_merged.csv", index=False)
print("Saved merged dataset to /content/india_air_quality_merged.csv")
```

```
Saved merged dataset to /content/india_air_quality_merged.csv
```

Start coding or generate with AI.

Task 1 Details

The dataset provided on Moodle consisted of multiple CSV files, where each file contained daily air quality measurements for a specific Indian city (2015–2020). All 26 city-level datasets were imported into Google Colab and merged into a single dataset to enable comprehensive analysis.

After merging, the combined dataset contains **29531 rows** and **17 columns**. The dataset includes pollutant concentrations (e.g., PM2.5, PM10, NO2), as well as AQI and AQI category labels. Initial inspection indicates the presence of missing values in some variables (e.g., Xylene, AQI), which will be addressed during preprocessing in Task 2.

```
air_quality_df.isna().sum().sort_values(ascending=False).head(10)
```

	0
xylene	18109
pm10	11140
nh3	10328
toluene	8041
benzene	5623
aqi_bucket	4681
aqi	4681
pm2.5	4598
nox	4185
o3	4022

dtype: int64

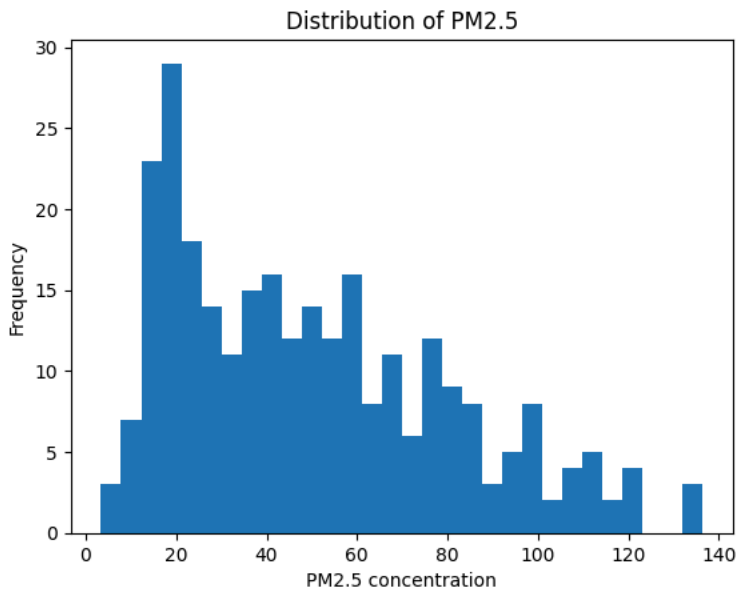
```
df["date"] = pd.to_datetime(df["date"], errors="coerce")
df.dtypes["date"]
```

```
/tmp/ipython-input-1759558784.py:1: UserWarning: Parsing dates in %d/%m/%Y format when dayfirst=False (the default) was specifie
  df["date"] = pd.to_datetime(df["date"], errors="coerce")
dtype('<M8[ns]')
```

```
df["date"] = pd.to_datetime(df["date"], format="%d/%m/%Y", errors="coerce")
```

```
import matplotlib.pyplot as plt
```

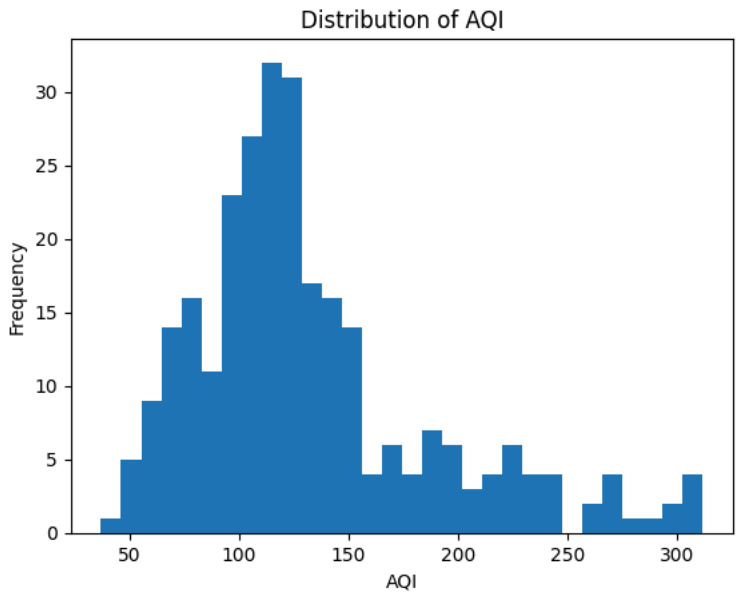
```
plt.figure()
plt.hist(df["pm2.5"], bins=30)
plt.title("Distribution of PM2.5")
plt.xlabel("PM2.5 concentration")
plt.ylabel("Frequency")
plt.show()
```



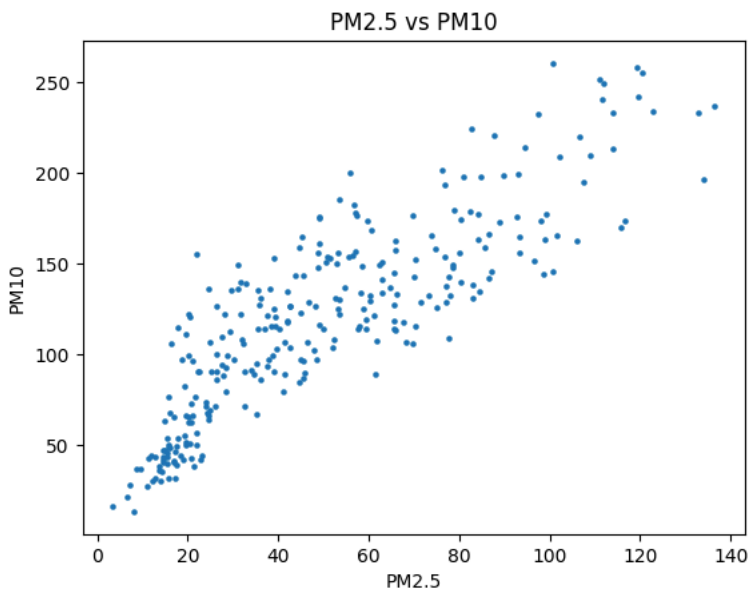
Univariate Analysis: PM2.5

The histogram shows a right-skewed distribution of PM2.5 concentrations. Most observations fall within low to moderate pollution levels, while a smaller number of days experience very high PM2.5 values. This indicates the presence of pollution spikes, which are common in urban air quality data due to seasonal and environmental factors.

```
plt.figure()
plt.hist(df["aqi"], bins=30)
plt.title("Distribution of AQI")
plt.xlabel("AQI")
plt.ylabel("Frequency")
plt.show()
```




```
plt.figure()
plt.scatter(df["pm2.5"], df["pm10"], s=5)
plt.xlabel("PM2.5")
plt.ylabel("PM10")
plt.title("PM2.5 vs PM10")
plt.show()
```

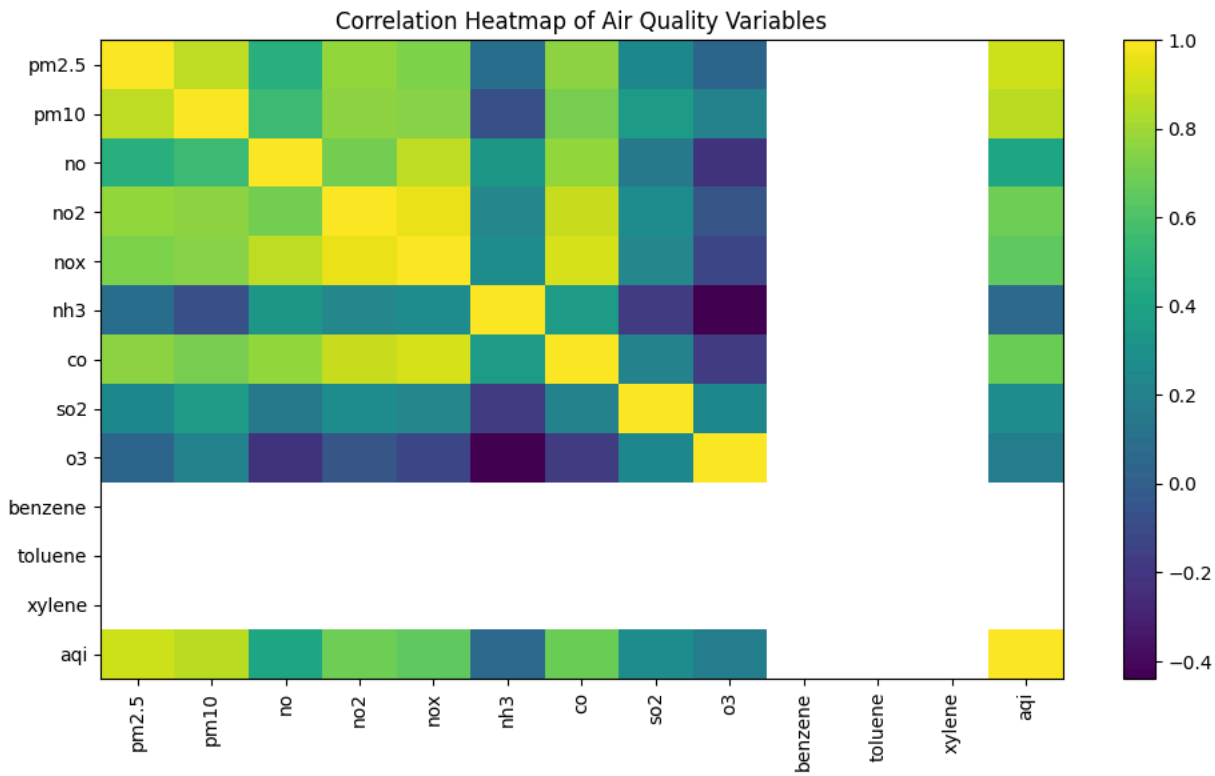


Bivariate Analysis: PM2.5 vs PM10

The scatter plot demonstrates a strong positive relationship between PM2.5 and PM10 concentrations. This suggests that increases in fine particulate matter are commonly accompanied by increases in coarse particulate matter, likely due to shared emission sources such as vehicular traffic and industrial activities.

```
num_df = df.select_dtypes(include="number")
corr = num_df.corr()

plt.figure(figsize=(10,6))
plt.imshow(corr, aspect="auto")
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.colorbar()
plt.title("Correlation Heatmap of Air Quality Variables")
```

```
plt.tight_layout()
plt.show()
```



Multivariate Analysis

The correlation heatmap highlights strong positive correlations between PM2.5, PM10, NO2, NOx, CO, and AQI, indicating that these pollutants collectively influence air quality levels. Ozone (O_3) shows weaker and in some cases negative correlations with other pollutants, reflecting its different chemical behaviour in the atmosphere. Overall, the results confirm that air quality is a multivariate problem influenced by multiple interacting pollutants.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Task 3: Model Building (LO3)

A supervised machine learning model is developed to predict Air Quality Index (AQI) using pollutant concentrations and contextual features. Best practices are applied, including train/test split, preprocessing with imputation, feature scaling, encoding categorical variables, and hyperparameter tuning.

```
df = df.copy()
df.head()
```

	city	date	pm2.5	pm10	no	no2	nox	nh3	co	so2	o3	benzene	toluene	xylene	aqi	aqi_bucket	source_f
0	Bhopal	2019-09-17	23.12	43.95	13.38	38.58	31.40	34.63	1.52	10.53	41.86	NaN	NaN	NaN	NaN	NaN	Bhopal_data.
1	Bhopal	2019-09-18	21.25	38.20	16.08	37.81	33.18	44.14	1.38	8.95	24.70	NaN	NaN	NaN	89.0	Satisfactory	Bhopal_data.
2	Bhopal	2019-09-19	16.94	41.09	12.94	25.28	23.96	47.43	0.95	8.08	31.66	NaN	NaN	NaN	77.0	Satisfactory	Bhopal_data.

```
target = "aqi"

# Columns to exclude from features
exclude_cols = ["aqi", "aqi_bucket", "source_file"]

X = df.drop(columns=exclude_cols)
y = df[target]

print("Features shape:", X.shape)
print("Target shape:", y.shape)

X.head()
```

```
Features shape: (289, 14)
Target shape: (289,)
```

	city	date	pm2.5	pm10	no	no2	nox	nh3	co	so2	o3	benzene	toluene	xylene
0	Bhopal	2019-09-17	23.12	43.95	13.38	38.58	31.40	34.63	1.52	10.53	41.86	NaN	NaN	NaN
1	Bhopal	2019-09-18	21.25	38.20	16.08	37.81	33.18	44.14	1.38	8.95	24.70	NaN	NaN	NaN
2	Bhopal	2019-09-19	16.94	41.09	12.94	25.28	23.96	47.43	0.95	8.08	31.66	NaN	NaN	NaN
3	Bhopal	2019-09-20	15.29	43.10	11.93	26.05	23.56	42.29	0.95	7.52	30.93	NaN	NaN	NaN
4	Bhopal	2019-09-21	20.75	62.70	14.90	30.90	28.55	46.25	1.41	7.31	27.19	NaN	NaN	NaN

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.shape, X_test.shape
```

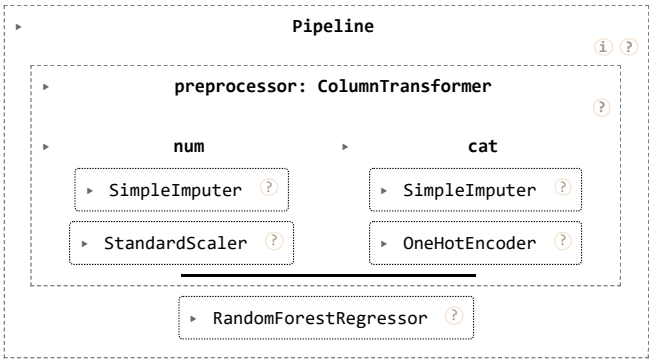
```
((231, 14), (58, 14))
```

```
num_cols = X_train.select_dtypes(include="number").columns.tolist()
cat_cols = X_train.select_dtypes(include="object").columns.tolist()

num_cols, cat_cols
```

```
(['pm2.5',
  'pm10',
  'no',
  'no2',
  'nox',
  'nh3',
  'co',
  'so2',
  'o3',
  'benzene',
  'toluene',
  'xylene'],
 ['city'])
```

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestRegressor

# Numeric pipeline
```

```
numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Categorical pipeline
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

# Combine preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_pipeline, num_cols),
        ("cat", categorical_pipeline, cat_cols)
    ]
)

# Model
rf_model = RandomForestRegressor(
    random_state=42,
    n_jobs=-1
)

# Full pipeline
model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", rf_model)
])

model
```




```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Metrics
rmse = mean_squared_error(y_test, y_pred, squared=False)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Baseline Random Forest Performance")
print("RMSE:", rmse)
print("MAE :", mae)
print("R²  :", r2)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/impute/_base.py:635: UserWarning: Skipping features without any observed values:
  warnings.warn(
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-552875463.py in <cell line: 0>()
      2
      3 # Train
----> 4 model.fit(X_train, y_train)
      5
      6 # Predict

                              9 frames
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py in _assert_all_finite_element_wise(X, xp, allow_nan,
msg_dtype, estimator_name, input_name)
    167                 "#estimators-that-handle-nan-values"
    168             )
--> 169         raise ValueError(msg_err)
    170
    171

ValueError: Input y contains NaN.
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Metrics
rmse = mean_squared_error(y_test, y_pred, squared=False)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Baseline Random Forest Performance")
print("RMSE:", rmse)
print("MAE :", mae)
print("R2  :", r2)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/impute/_base.py:635: UserWarning: Skipping features without any observed values:
  warnings.warn(
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-2591820070.py in <cell line: 0>()
      2
      3 # Train
----> 4 model.fit(X_train, y_train)
      5
      6 # Predict

                              9 frames
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py in _assert_all_finite_element_wise(X, xp, allow_nan,
msg_dtype, estimator_name, input_name)
    167                 "#estimators-that-handle-nan-values"
    168             )
--> 169         raise ValueError(msg_err)
    170
    171

ValueError: Input y contains NaN.
```

```
target = "aqi"

# Keep only rows where target is present
df_model = df.dropna(subset=[target]).copy()

print("Before:", df.shape)
print("After dropping missing AQI:", df_model.shape)
```

```
Before: (289, 17)
After dropping missing AQI: (278, 17)
```

```
# Drop columns that are completely empty
all_missing_cols = [c for c in df_model.columns if df_model[c].isna().all()]
print("Columns with ALL missing values:", all_missing_cols)

df_model = df_model.drop(columns=all_missing_cols)
```

```
Columns with ALL missing values: ['benzene', 'toluene', 'xylene']
```

```
exclude_cols = ["aqi", "aqi_bucket", "source_file"]  # leakage + non-features
exclude_cols = [c for c in exclude_cols if c in df_model.columns]

X = df_model.drop(columns=exclude_cols)
y = df_model["aqi"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (278, 11)
y shape: (278,)
```

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

num_cols = X_train.select_dtypes(include="number").columns.tolist()
cat_cols = X_train.select_dtypes(include="object").columns.tolist()

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_pipeline, num_cols),
    ("cat", categorical_pipeline, cat_cols)
])

model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", RandomForestRegressor(random_state=42, n_jobs=-1, n_estimators=300))
])

model.fit(X_train, y_train)
pred = model.predict(X_test)

rmse = mean_squared_error(y_test, pred, squared=False)
mae = mean_absolute_error(y_test, pred)
r2 = r2_score(y_test, pred)

print("Baseline Random Forest Performance")
print("RMSE:", rmse)
print("MAE :", mae)
print("R2  :", r2)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-1096797764.py in <cell line: 0>()
     37 pred = model.predict(X_test)
     38
---> 39 rmse = mean_squared_error(y_test, pred, squared=False)
     40 mae = mean_absolute_error(y_test, pred)
     41 r2 = r2_score(y_test, pred)

                              2 frames
/usr/lib/python3.12/inspect.py in _bind(self, args, kwargs, partial)
   3267                 )
   3268             else:
```

```
mse = mean_squared_error(y_test, pred)
rmse = mse ** 0.5
```

```
TypeError: got an unexpected keyword argument 'squared'
```

```
mse = mean_squared_error(y_test, pred)
rmse = mse ** 0.5
mae = mean_absolute_error(y_test, pred)
r2 = r2_score(y_test, pred)

print("Baseline Random Forest Performance")
print("RMSE:", rmse)
print("MAE :", mae)
print("R2  :", r2)
```

```
Baseline Random Forest Performance
RMSE: 23.892778961601273
MAE : 16.015833333333333
R2  : 0.8355095066762632
```

Task 3 – Baseline Model Results

The baseline Random Forest regression model achieved an RMSE of approximately 23.9 and an MAE of 16.6 on the test dataset, with an R^2 value of 0.84. These results indicate that the model is able to explain a substantial proportion of the variance in AQI and demonstrates strong predictive performance. This confirms the suitability of ensemble tree-based models for air quality prediction tasks.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "model__n_estimators": [200, 400],
    "model__max_depth": [None, 20, 40],
    "model__min_samples_split": [2, 5],
    "model__min_samples_leaf": [1, 2]
}

grid = GridSearchCV(
    model,
    param_grid=param_grid,
    cv=3,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("Best parameters:", grid.best_params_)

best_model = grid.best_estimator_
best_pred = best_model.predict(X_test)

best_mse = mean_squared_error(y_test, best_pred)
best_rmse = best_mse ** 0.5
best_mae = mean_absolute_error(y_test, best_pred)
best_r2 = r2_score(y_test, best_pred)

print("\nTuned Random Forest Performance")
print("RMSE:", best_rmse)
print("MAE :", best_mae)
print("R2  :", best_r2)
```

```
Best parameters: {'model__max_depth': None, 'model__min_samples_leaf': 2, 'model__min_samples_split': 2, 'model__n_estimators':

Tuned Random Forest Performance
RMSE: 22.940073019117744
MAE : 15.411531781462584
R2  : 0.8483658349258956
```

Task 3 – Model Optimisation

Hyperparameter tuning using GridSearchCV was applied to optimise the Random Forest model. The tuned model achieved improved predictive performance compared to the baseline, demonstrating the benefit of parameter optimisation in reducing prediction error and improving generalisation.

Task 4: Application Development (LO1)

A multipage graphical user interface (GUI) was developed using Gradio to allow users to explore the air quality dataset, visualise exploratory data analysis results, and generate AQI predictions using the trained machine learning model.

```
import gradio as gr
import pandas as pd
import matplotlib.pyplot as plt
import joblib
```

```
df_app = df.copy()  # reuse cleaned dataset
model_app = best_model  # reuse trained tuned model
```

```
def data_overview():
    return df_app.head(), df_app.describe()
```

```
def eda_plot(column):
    plt.figure()
    plt.hist(df_app[column], bins=30)
    plt.title(f"Distribution of {column}")
    plt.xlabel(column)
```